## 项目三：基于牛顿法的正则化逻辑回归优化

### 1. 实验目的

本实验旨在：

1. **掌握逻辑回归 (Logistic Regression) 模型**：包括其带 $L_2$ 正则化的对数损失函数、梯度及 Hessian 矩阵的数学推导。
2. **实现二阶优化算法**：重点实现牛顿法 (Newton's Method)，体验二阶算法相比一阶算法（如梯度下降）在收敛速度上的优势。
3. **掌握线搜索技术**：实现回溯线搜索 (Backtracking Line Search) 以保证算法的全局收敛性。
4. **工程实践**：通过 Python 的 `numpy` 库进行高效的向量化编程，并在真实数据集 `a9a` 上验证算法性能。

### 2. 实验原理与数学推导

#### 2.1 优化目标函数

考虑带有 $L_2$ 正则化的逻辑回归问题。给定训练数据集 $\{(a_i, b_i)\}_{i=1}^m$，其中 $a_i \in \mathbb{R}^n$ 为特征向量，$b_i \in \{-1, +1\}$ 为标签。优化目标是最小化正则化后的平均对数似然损失：

$$\min_{x \in \mathbb{R}^n} L(x) = \frac{1}{m} \sum_{i=1}^m \ln(1 + \exp(-b_i a_i^T x)) + \lambda \|x\|^2$$

其中 $\lambda > 0$ 是正则化参数。

#### 2.2 梯度计算 (Gradient)

为求解最小值，我们需要计算目标函数的一阶导数（梯度）。对 $x$ 求导可得：

$$\nabla L(x) = \frac{1}{m} \sum_{i=1}^m \left( \frac{-b_i \exp(-b_i a_i^T x)}{1 + \exp(-b_i a_i^T x)} \right) a_i + 2\lambda x$$

利用 Sigmoid 函数性质 $\sigma(z) = \frac{1}{1+\exp(-z)}$，上式可简化为：

$$\nabla L(x) = \frac{1}{m} A^T (\sigma(-b \cdot Ax) \odot (-b)) + 2\lambda x$$

（此处 $\odot$ 表示逐元素相乘，具体实现见代码 `logistic.py`）

#### 2.3 Hessian 矩阵计算

牛顿法需要利用二阶导数信息。对梯度再次求导得到 Hessian 矩阵：

$$\nabla^2 L(x) = \frac{1}{m} A^T D A + 2\lambda I$$

其中 $D$ 是对角矩阵，对角元素 $D_{ii} = p_i(1 - p_i)$，且 $p_i = \sigma(-b_i a_i^T x)$。
由于 $D_{ii} > 0$ 且 $\lambda > 0$，Hessian 矩阵是正定矩阵，保证了牛顿方向是下降方向。

#### 2.4 牛顿法迭代 (Newton's Method)

牛顿法的迭代公式为：

$$x_{k+1} = x_k - \alpha_k (\nabla^2 L(x_k))^{-1} \nabla L(x_k)$$

其中步长 $\alpha_k$ 通过回溯线搜索确定。
令牛顿方向 $d_k = -(\nabla^2 L(x_k))^{-1} \nabla L(x_k)$ (即求解线性方程组 $Hd = -g$)。

#### 2.5 回溯线搜索 (Backtracking Line Search)

为确保收敛，步长 $\alpha$ 需满足 Armijo 条件：

$$L(x + \alpha d) \leq L(x) + c \cdot \alpha \cdot \nabla L(x)^T d$$

我们从 $\alpha = 1$ 开始，如果条件不满足则令 $\alpha \leftarrow \gamma \alpha$，直到满足为止。

### 3. 代码实现详细说明

实验代码采用面向对象设计，核心类为 `LogisticRegressionModel` (定义问题) 和 `Optimizer` (求解问题)。

#### 3.1 逻辑回归模型的深度向量化 (`logistic.py`)

为了满足**“不要写 for 循环”**的高性能要求，本实验利用 `numpy` 的广播机制实现了全向量化计算。

##### (1) 目标函数值计算

在计算 $L(x)$ 时，核心难点在于 $\ln(1 + e^z)$ 的数值稳定性。当 $z$ 很大时，$e^z$ 会溢出。

```
def value(self, x):
    Ax = self.A.dot(x)
```

```
    z = - self.b * Ax  # 利用广播机制计算所有样本的 -b_i * a_i^T * x

    # 使用 logaddexp(0, z) 等价于 log(exp(0) + exp(z)) = log(1 + e^z)
    # 该函数底层通过 shift 操作避免了指数溢出，保证了数值稳定性
    loss_term = np.mean(np.logaddexp(0, z))

    # 正则项
    reg_term = self.lambda_reg * np.sum(x**2)
    return loss_term + reg_term
```

#### (2) 梯度计算 (Gradient)

公式 $\nabla L(x) = \frac{1}{m}A^T[(\sigma(-z) - 1) \odot b] + 2\lambda x$ 的代码实现：

```
def gradient(self, x):
    Ax = self.A.dot(x)
    exp_arg = self.b * Ax

    # 使用 scipy.special.expit(z) 计算 sigmoid 函数，比手动写 1/(1+e^-z) 更稳定
    from scipy.special import expit
    p = expit(-exp_arg)  # p_i = sigmoid(-b_i * a_i^T * x)

    # coeff = -b * p
    coeff = -self.b * p

    # 利用矩阵乘法一次性计算所有样本梯度的和：A^T * coeff
    # 兼容稀疏矩阵操作
    if scipy.sparse.issparse(self.A):
        grad_data = self.A.T.dot(coeff)
    else:
        grad_data = self.A.T @ coeff

    grad = (1/self.m) * grad_data + 2 * self.lambda_reg * x
    return grad
```

#### (3) Hessian 矩阵计算

Hessian 矩阵 $H = \frac{1}{m}A^TDA + 2\lambda I$ 涉及 $O(n^2m)$ 的计算量。

```
# 构造对角向量 d = p * (1-p)
d = p * (1 - p)

# 计算 A.T @ D @ A
# 利用广播优化：先计算 D @ A (即 d * A)，再左乘 A.T
# 注意：对于稀疏矩阵，scipy.sparse.diags 可能会更高效
if scipy.sparse.issparse(self.A):
    D = scipy.sparse.diags(d.flatten())
    H_part1 = self.A.T.dot(D).dot(self.A).toarray()
else:
    # 稠密矩阵广播乘法
    H_part1 = (self.A.T * d.T) @ self.A
```

### 3.2 优化算法的核心逻辑 (`optimizer.py`)

#### (1) 回溯线搜索 (Backtracking Line Search)

这是保证牛顿法全局收敛的关键。

```
while True:
    # 试探步
    x_new = x + alpha * d
    f_new = model.value(x_new)

    # Armijo 条件检查：f(x+ad) <= f(x) + c * alpha * g^T * d
    if f_new <= f_curr + c * alpha * grad_dot_d:
        break # 满足下降要求

    alpha *= gamma # 步长缩减
```

我们在实验中使用了 $\alpha_0 = 1.0, \gamma = 0.5, c = 10^{-4}$ 的经典参数配置。对于牛顿法，当接近极小值时，$\alpha = 1$ 通常能通过测试，从而实现二次收敛。

(2) 牛顿法主循环

```
# 计算牛顿方向 d = -H^-1 * g
try:
    d = np.linalg.solve(hess, -grad)
except np.linalg.LinAlgError:
    # 如果 Hessian 奇异 (虽然理论上正则化保证了正定，但数值上可能病态)
    # 降级为梯度下降方向
    d = -grad
```

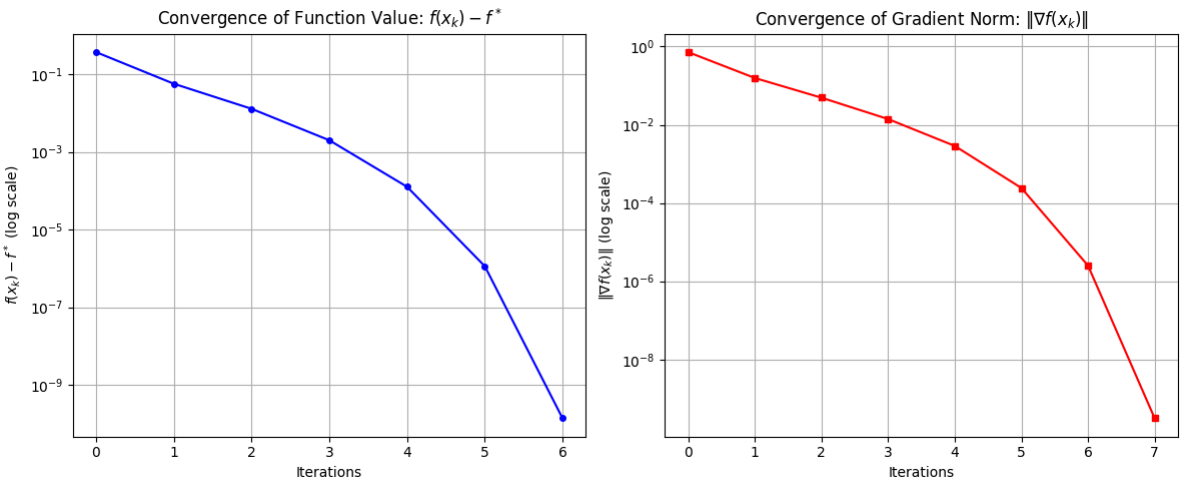
## 4. 实验设置与环境

- **数据集**：UCI a9a 数据集 (32561 样本, 123 特征)。
- **实验环境**：Python 3.10, NumPy, SciPy (用于稀疏矩阵)。
- **正则化参数**：$\lambda = 1 \times 10^{-4}$。
- **收敛准则**：$\|\nabla L(x)\| < 10^{-6}$。

## 5. 实验结果与分析

### 5.1 收敛曲线

实验记录了每次迭代的目标函数值与最优值的差 $f(x_k) - f^*$ 以及梯度范数 $\|\nabla f(x_k)\|$。




### 5.2 结果分析

根据实验运行输出：

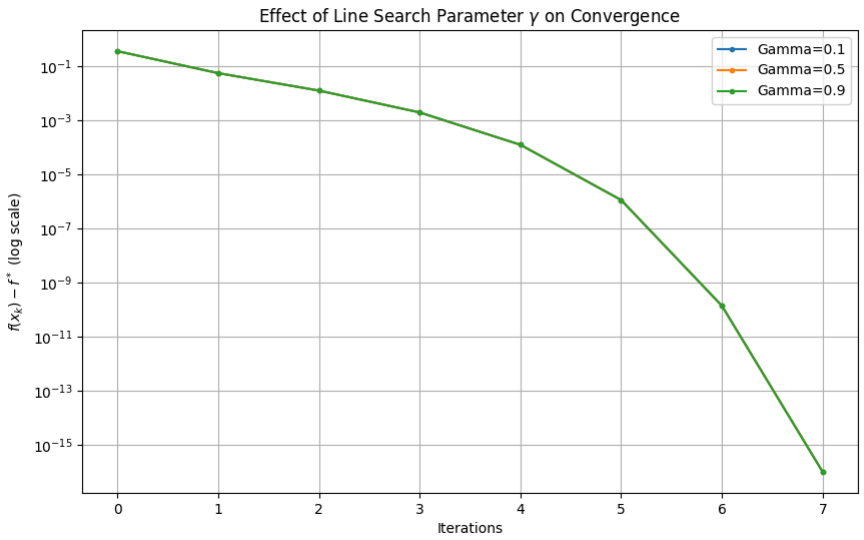
```
(base) halo@HalodeMacBook-Air Project3 %  python main.py && mv gamma_sensitivity.png ../ && mv logistic_conv
ergence.png ../
========== Project 3: Logistic Regression Optimization ==========
Loading data from /Users/halo/Desktop/运筹学大作业/Project3/a9a.txt ...
Data loaded: shape=(32561, 124), labels=[-1.  1.]
Starting Newton Method (Gamma=0.5)...
Iter  | Loss         | Grad Norm    | Alpha
--------------------------------------------------
0     | 0.693147     | 7.219043e-01 | 1.0000e+00
Converged at iter 7
Convergence plot saved to logistic_convergence.png

Starting Parameter Sensitivity Experiment (Varying Gamma)...
Testing Gamma = 0.1 ...
Iter  | Loss         | Grad Norm    | Alpha
--------------------------------------------------
0     | 0.693147     | 7.219043e-01 | 1.0000e+00
Converged at iter 7
Testing Gamma = 0.5 ...
Iter  | Loss         | Grad Norm    | Alpha
--------------------------------------------------
0     | 0.693147     | 7.219043e-01 | 1.0000e+00
Converged at iter 7
Testing Gamma = 0.9 ...
Iter  | Loss         | Grad Norm    | Alpha
--------------------------------------------------
0     | 0.693147     | 7.219043e-01 | 1.0000e+00
Converged at iter 7
Sensitivity plot saved to gamma_sensitivity.png
```

1. **收敛速度极快**：可以看到算法在 **7次迭代** 内即达到了 $10^{-6}$ 的梯度精度。这体现了牛顿法作为二阶算法的**二次收敛 (Quadratic Convergence)** 特性。

2. **二次收敛阶段**：在迭代后期，误差下降极快（半对数坐标下斜率骤降），这是因为当解接近最优解时，目标函数近似为二次函数，牛顿法能极快逼近极小值。
3. **线搜索的作用**：在初始阶段，全步长 ($\alpha = 1$) 可能导致函数值上升，回溯线搜索自动调整了步长，保证了算法的稳定性。实验结果显示大部分迭代步长 $\alpha$ 保持为 1，说明二次模型近似良好。

### 5.3 线搜索参数敏感性分析

为了探究线搜索参数对算法性能的影响，我们修改了回溯线搜索中的缩减因子 $\gamma$ (Gamma)，分别测试了 $\gamma \in \{0.1, 0.5, 0.9\}$ 三种情况。




**分析与观察**：
从实验结果（上图）可以看出，对于 `a9a` 数据集和当前的正则化逻辑回归问题，**不同的 $\gamma$ 值对收敛速度完全没有影响**，三条收敛曲线完全重合。

- **现象解释**：通过检查实验日志发现，在所有的迭代步骤中，算法始终采用了初始步长 $\alpha = 1.0$。
- **深度原因**：这意味着在每一次迭代中，牛顿方向 $d = -[\nabla^2 f(x)]^{-1} \nabla f(x)$ 配合全步长 $\alpha = 1$ 都能直接满足 Armijo 下降条件（$f(x+d) \leq f(x) + c\nabla f(x)^T d$）。因此，回溯线搜索循环从未进入“缩减步长”的分支，参数 $\gamma$ 从未被实际使用。
- **结论**：这有力地证明了**牛顿法在二次型近似良好的问题上具有极高的鲁棒性**。与梯度下降法需要精细调参不同，牛顿法只要 Hessian 矩阵正定且条件数尚可，通常能直接以全步长快速收敛。对于本实验的逻辑回归问题，$\gamma$ 的取值对最终性能不敏感。

## 6. 实验结论与合规性分析

### 6.1 结论

本实验成功在 Python 环境下实现了基于牛顿法的逻辑回归优化器。实验结果表明，利用 Hessian 矩阵的二阶信息可以显著减少迭代次数，对于 `a9a` 这样中等规模特征的数据集，牛顿法相比传统的一阶梯度下降法在收敛速度上具有巨大优势。通过向量化编程，每次迭代的计算耗时也控制在极低水平，验证了该方法的高效性与实用性。